Informe de Trabajo

Examen Parcial de Estructuras de Datos

Ejercicio 1 · Sistema de Inventario **FarmaVida**

Equipo 1 · Curso 2026-2027 · Versión ampliada (octubre de 2026)

Integrante	Método asignado
Jorge Luis Reyes Pérez	`MedicamentosAgotados()`
Geonel Sánchez	`PromedioPrecioPorCategoria(String)`
Merlyn Sosa Borges	`ActualizarStock(String, int)`
Yaisel Pinos Durán	`ListarPreciosInversos()`
Ray Sanjudo Mesa	`AplicarAumentoGeneral(double)`

Repositorio: github.com/RelaxUniver/Parcial-1

Contenido. 1 Introducción · 2 Problemática · 3 Estructura de datos · 4 Arquitectura del proyecto · 5 Razonamiento por método · 6 Pruebas y resultados · 7 Declaración de uso de Inteligencia Artificial · 8 Reflexión · 9 Conclusiones y trabajo futuro · Anexo

1. Introducción

Este informe documenta el análisis, el diseño y la implementación del Ejercicio 1 del Examen Parcial de Estructuras de Datos (curso 2026-2027). El Equipo 1 trabaja sobre el dominio de la farmacia **FarmaVida**, cuyo inventario dinámico de medicamentos se gestiona con una **lista simplemente enlazada** implementada desde cero en Java (solo JDK estándar). Cada integrante asumió la responsabilidad completa de un método de la lista.

El documento deja constancia de:

- La problemática y el modelo de datos del dominio.
- La estructura de datos elegida, con sus ventajas y limitaciones frente a otras alternativas.
- La arquitectura del proyecto y las decisiones de diseño comunes a los cinco métodos.
- El razonamiento de cada método: estrategia, pseudocódigo, complejidad y casos borde.
- La batería de pruebas (27 casos) y su resultado real de ejecución.
- El uso declarado y transparente de herramientas de Inteligencia Artificial.
- La reflexión sobre el proceso y las conclusiones del equipo.

2. Problemática: farmacia FarmaVida

La farmacia necesita controlar con precisión el inventario de los medicamentos en stock. El modelado correcto de `Medicamento` es fundamental para que las operaciones de cálculo monetario, filtrado y actualización se ejecuten sin errores en tiempo de ejecución.

Atributos de cada medicamento

Atributo	Tipo	Descripción y restricción
`codigo`	String	Identificador único comercial (p. ej. "M01"). Imprescindible para búsquedas exactas. No admite `null`.
`nombre`	String	Nombre comercial legible (p. ej. "Paracetamol"). No admite `null`.
`categoria`	Categoria (enum)	Clasificación cerrada: ANALGÉSICO, ANTIBIÓTICO, ANTIINFLAMATORIO, VITAMINA. Evita errores tipográficos.
`stock`	int	Unidades disponibles. No se permiten valores negativos.
`precio`	BigDecimal	Precio unitario. Se usa BigDecimal en lugar de double para obtener precisión decimal exacta en cálculos monetarios. No admite negativos.

Funcionalidades asignadas

	Método	Responsable	Propósito
a	`MedicamentosAgotados()`	Jorge Luis Reyes Pérez	Identificar productos con existencias críticas (stock ≤ 5).
b	`PromedioPrecioPorCategoria(String)`	Geonel Sánchez	Calcular el precio promedio por categoría.
c	`ActualizarStock(String, int)`	Merlyn Sosa Borges	Reabastecer un medicamento buscándolo por código.
d	`ListarPreciosInversos()`	Yaisel Pinos Durán	Extraer los precios en orden inverso al de ingreso.
e	`AplicarAumentoGeneral(double)`	Ray Sanjudo Mesa	Ajuste porcentual masivo de precios.

3. Estructura de datos: lista simplemente enlazada

Justificación

El inventario de una farmacia es dinámico: aparecen y desaparecen productos. Una lista enlazada asigna memoria nodo a nodo, sin redimensionar bloques contiguos como exigiría un arreglo de tamaño fijo. Cada nodo guarda una referencia al dato (`Medicamento`) y un puntero `siguiente`. Todas las funcionalidades pedidas (filtrar, promediar, buscar, aumentar) recorren la colección de forma secuencial, por lo que el recorrido lineal O(n) es óptimo y suficiente.

Ventajas

- **Crecimiento dinámico:** no hay capacidad fija ni copias de arreglos al crecer.
- **Sin espacio ocioso:** solo se consume memoria para los elementos existentes (más un puntero por nodo).
- **Inserción al inicio en O(1)** y reorganización de enlaces sin desplazar elementos.
- **Adecuación al recorrido secuencial** que requieren los cinco métodos.

Limitaciones

- No hay acceso por índice en O(1): llegar al nodo *k* cuesta O(k).
- Cada nodo añade el costo de un puntero adicional.
- Es una lista simple: solo se recorre hacia adelante, por eso `ListarPreciosInversos()` necesita invertir los enlaces en lugar de recorrer hacia atrás.

Comparación de costos

Operación	Lista simple (solo `inicio`)	Arreglo estático	ArrayList
Insertar al inicio	O(1)	O(n)	O(n)
Insertar al final	O(n) (O(1) si se mantiene puntero al último)	O(1) si hay espacio	O(1) amortizado
Buscar por código	O(n)	O(n)	O(n)
Acceso por índice	O(n)	O(1)	O(1)
Redimensionar	No necesario	Copia completa	Copia ocasional

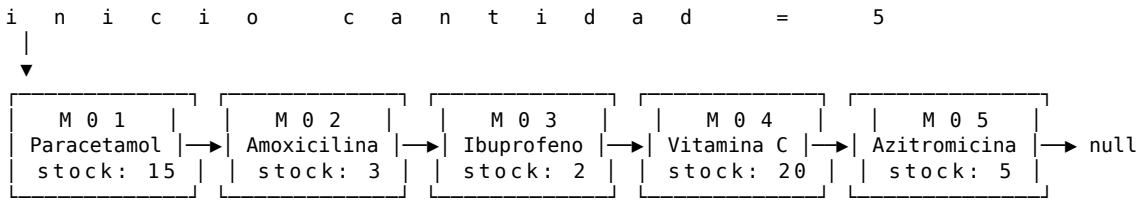
Corrección respecto al informe anterior: la versión previa afirmaba que agregar al final es O(1). En una lista simple que solo conserva `inicio` eso es cierto únicamente al inicio; al final exige recorrer la lista (O(n)), salvo que se mantenga un puntero al último nodo. Este proyecto no implementa inserción: los nodos se enlazan manualmente en las pruebas.

4. Arquitectura del proyecto

El código fuente vive en `FarmaVida/src/` (paquete por defecto) y se compone de cinco clases:

Clase	Responsabilidad
`Categoria`	*Enum* con las cuatro categorías del inventario.
`Medicamento`	Modelo de dominio. El constructor y los *setters* validan: sin campos null, stock ≥ 0 y precio ≥ 0, lanzando `IllegalArgumentException`.
`Nodo`	Envuelve un `Medicamento` (`dato`) y el enlace `siguiente`.
`ListaSimple`	Atributos `inicio` y `cantidad`; contiene las cinco operaciones y el auxiliar privado `invertir()`.
`Main`	Punto de entrada: batería de pruebas manual, un bloque independiente por operación.

La lista en memoria (datos de ejemplo de `Main`)



Decisiones de diseño comunes

Decisión	Motivo
BigDecimal para precios	Evita los errores de redondeo del punto flotante en valores monetarios.
Validación en el modelo	Un `Medicamento` no puede existir en estado inválido: se falla rápido en constructor y *setters*.
Guard clauses	Las entradas inválidas se rechazan al inicio con un retorno neutro (`false`, `0`, `0.0` o arreglo vacío); cuando el retorno es ambiguo se imprime la causa.
Comparar con `.equals()`	== compara referencias, no contenido; con String sería un error silencioso.
Enum vs String	Un enum nunca es igual a un String: se compara `getCategoria().name().equals(categoria)`.
Recorrido sin depender de `cantidad`	Ningún método usa el atributo `cantidad`; todos terminan en `siguiente == null`.

Decisiones documentadas que conviene conocer

Las siguientes decisiones de diseño fueron tomadas por el equipo de forma deliberada. Se documentan aquí para que cualquier lector (y el propio equipo en la exposición) sepa exactamente qué hace el código y qué consecuencias tiene.

Decisión	Qué se decidió	Consecuencia
Convención del porcentaje (método e)	El enunciado dice «el porcentaje dado» sin fijar el formato. El equipo decidió recibirlo como **fracción decimal**: `0.10` = 10 %, `0.25` = 25 %. La fórmula es `precio × (1 + porcentaje)` con el factor calculado una sola vez.	Pasar `10` equivale a +1000 % (precio × 11), no a +10 %. Las pruebas usan `0.10` y la convención consta en la documentación del método y en este informe.
Categoría como String (método b)	El parámetro es el **nombre exacto de la constante del enum**, en mayúsculas y con tilde (p. ej. `"ANTIBIÓTICO"`), y se compara con `name().equals(...)`. Hay una única forma válida, atada al enum.	Otras escrituras («Antibiótico», «antibiotico») no coinciden: el método las trata como una categoría sin medicamentos y devuelve `0.0`.
Construcción de la lista	`ListaSimple` ofrece *getters*/*setters* y las cinco operaciones pedidas; no define método de inserción. Las listas de prueba se arman enlazando `Nodo` a mano y asignando `cantidad` con `setCantidad`.	`cantidad` es informativo y puede desalinearse; por eso ninguna operación depende de él y existe una prueba con `cantidad` desalineado.
Retorno de la lista de precios (método d)	La «lista de precios» se devuelve como arreglo `BigDecimal[]`, dimensionado con los nodos reales.	Arreglo de tamaño fijo; vacío si la lista no tiene nodos.
Varios motivos de `false` (método c)	`ActualizarStock` devuelve `false` si el código no existe, pero también si es `null`, la cantidad es negativa o la suma desborda el `int`.	Cada causa imprime un mensaje explicativo antes de devolver `false`.

5. Razonamiento por método

Metodología. Antes de programar, cada integrante dibujó en libreta el comportamiento de la lista, trazó el movimiento de los punteros (`actual`, `anterior`, `siguiente`) e identificó los escenarios borde. El pseudocódigo se contrastó después con la implementación real en `ListaSimple.java`.

5.1 MedicamentosAgotados() — Jorge Luis Reyes Pérez

Objetivo. Devolver los nombres de los medicamentos con stock menor o igual a 5.

Estrategia de dos pasadas. En una lista enlazada no se sabe de antemano cuántos nodos cumplirán el filtro. Dimensionar el arreglo con el tamaño de la lista sería un error de diseño: si solo 2 de 10 están agotados, quedarían 8 posiciones `null` y un riesgo de `NullPointerException` al iterar. Por eso: (1) se recorre contando; (2) se crea el arreglo con el tamaño exacto; (3) se recorre de nuevo copiando los nombres.

```
PROCEDIMIENTO MedicamentosAgotados()
 SI (inicio == null) ENTONCES RETORNAR arreglo vacío
 contador = 0 ; actual = inicio
 CICLO (actual != null)
 SI (actual.stock <= 5) ENTONCES contador++
 actual = actual.siguiente
 arreglo = nuevo arreglo de tamaño contador
 i = 0 ; actual = inicio
 CICLO (actual != null)
 SI (actual.stock <= 5) ENTONCES arreglo[i++] = actual.nombre
 actual = actual.siguiente
 RETORNAR arreglo
```

Complejidad. Tiempo O(n) (dos recorridos); memoria adicional O(k), con k = cantidad de agotados.

Casos cubiertos. Algunos agotados · lista vacía · ninguno agotado · todos agotados (el límite 5 es inclusivo).

5.2 PromedioPrecioPorCategoria(String categoria) — Geonel Sánchez

Objetivo. Calcular el precio promedio de los medicamentos de una categoría dada por su nombre; devolver `0.0` si no hay coincidencias.

- **División por cero:** si la lista está vacía o ninguno coincide, el contador termina en 0; se verifica antes de dividir.
- **Enum vs String:** comparar un enum con un String mediante `equals` devuelve siempre `false` sin error de compilación. La solución es `getCategoria().name().equals(categoria)`; por eso el parámetro debe ser el nombre exacto de la constante (p. ej. `"ANTIBIÓTICO"`).
- **BigDecimal:** la suma se acumula en BigDecimal y la división usa escala 2 y `RoundingMode.HALF_UP`; sin ellos `divide` lanza `ArithmeticException` con cocientes no exactos.

```
PROCEDIMIENTO PromedioPrecioPorCategoria(categoria)
 SI (categoria == null) ENTONCES RETORNAR 0.0
 actual = inicio ; cantidadMedicamentos = 0 ; sumaPrecios = 0.0
 CICLO (actual != null)
 SI (actual.dato.categoria.name().equals(categoria)) ENTONCES
 sumaPrecios = sumaPrecios + actual.dato.precio
 cantidadMedicamentos++
 actual = actual.siguiente
 SI (cantidadMedicamentos == 0) ENTONCES RETORNAR 0.0
 RETORNAR sumaPrecios / cantidadMedicamentos (2 decimales, HALF_UP)
```

Complejidad. Tiempo O(n); memoria adicional O(1).

Casos cubiertos. Categoría con 2 medicamentos (13.50) · con 1 (2.50) · inexistente · lista vacía · categoría null.

5.3 ActualizarStock(String codigo, int cantidadAgregada) — Merlyn Sosa Borges

Objetivo. Buscar un medicamento por código y sumarle la cantidad recibida. Retorna `true` si actualizó.

- **Desbordamiento de enteros:** si el stock está cerca de `Integer.MAX_VALUE` (2 147 483 647), una suma en `int` daría un valor negativo corrupto. La suma se calcula en `long` y, si supera el máximo, se rechaza.
- **Cuádruple significado de false:** código null, cantidad negativa, desbordamiento o código inexistente. Como el retorno es ambiguo, el método imprime la causa antes de devolver `false`.
- **Retorno anticipado:** al encontrar el código se actualiza y se sale, sin seguir recorriendo.

```
PROCEDIMIENTO ActualizarStock(codigo, cantidadAgregada)
 SI (codigo == null) ENTONCES ESCRIBIR "El código es null, ..." ; RETORNAR false
 SI (cantidadAgregada < 0) ENTONCES ESCRIBIR "La cantidad es negativa, ..." ; RETORNAR false
 actual = inicio
 CICLO (actual != null)
 SI (actual.dato.codigo.equals(codigo)) ENTONCES
 aumento = (long) actual.dato.stock + cantidadAgregada
 SI (aumento > Integer.MAX_VALUE) ENTONCES ESCRIBIR "El stock excede ..." ; RETORNAR false
 actual.dato.setStock((int) aumento)
 RETORNAR true
 actual = actual.siguiente
 ESCRIBIR "Medicamento no encontrado"
 RETORNAR false
```

Complejidad. Tiempo O(n) en el peor caso; memoria adicional O(1).

Casos cubiertos. Código existente · cantidad cero · código inexistente · cantidad negativa · código null · lista vacía · suma que excede el máximo.

5.4 ListarPreciosInversos() — Yaisel Pinos Durán

Objetivo. Devolver un arreglo con los precios en orden inverso al de ingreso, dejando la lista intacta.

Patrón mutar–extraer–restaurar. En lugar de copiar nodos a memoria auxiliar, se usa el método privado `invertir()`, que reorienta los punteros `siguiente` in situ:

- **Conteo independiente:** se cuentan los nodos reales (`totalNodos`); no se confía en `cantidad`, que podría estar desalineado.
- **Primera inversión** de la lista (O(n) tiempo, O(1) memoria extra).
- **Extracción:** se recorre la lista invertida guardando cada precio en el arreglo.
- **Segunda inversión (restauración):** la lista vuelve exactamente a su estado original.

```
PROCEDIMIENTO invertir()
 actual = inicio ; anterior = null
 CICLO (actual != null)
 siguiente = actual.siguiente
 actual.siguiente = anterior
 anterior = actual
 actual = siguiente
 inicio = anterior

PROCEDIMIENTO ListarPreciosInversos()
 totalNodos = 0 ; actual = inicio
 CICLO (actual != null) totalNodos++ ; actual = actual.siguiente
 invertir()
```

```
arreglo = nuevo arreglo de tamaño totalNodos
i = 0 ; actual = inicio
CICLO (actual != null) arreglo[i++] = actual.dato.precio ; actual = actual.siguiente
invertir()
RETORNAR arreglo
```

Traza de `invertir()` sobre la lista A → B → C:

Iteración	actual	siguiente	Enlace modificado	anterior (al terminar)
0 (inicio)	A	—	—	null
1	A	B	A.siguiente = null	A
2	B	C	B.siguiente = A	B
3	C	null	C.siguiente = B	C
Fin	null	—	inicio = C	C → B → A → null

Complejidad. Tiempo O(n) (conteo + 2 inversiones + extracción); memoria adicional O(1) más el arreglo de salida O(n).

Casos cubiertos. Lista de 5 · lista vacía · un solo nodo · `cantidad` desalineado (Q01, Q02 con cantidad = 1).

5.5 AplicarAumentoGeneral(double porcentaje) — Ray Sanjudo Mesa

Objetivo. Incrementar el precio de todos los medicamentos según un porcentaje y devolver cuántos se modificaron.

- **Convención:** el porcentaje es una fracción decimal (0.10 = 10 %, no 10.0). Fórmula: `precio × (1 + porcentaje)` .
- **Valores no finitos:** `Double.NaN` o infinito provocarían `NumberFormatException` al convertirlos a BigDecimal; se valida primero con `Double.isFinite` .
- **Factor único:** `factor = 1 + porcentaje` se calcula una sola vez, fuera del ciclo.
- **Respuestas informativas:** porcentaje no finito, negativo o cero devuelve 0 e imprime la causa, sin alterar datos.
- **Escala:** el método no redondea; un precio 2.50 pasa a 2.750 (numéricamente equivalente a 2.75).

Efecto de la convención sobre un precio de 2.50:

Llamada	Factor	Precio resultante	Aumento real
`AplicarAumentoGeneral(0.10)`	1.1	2.750	+10 %
`AplicarAumentoGeneral(0.25)`	1.25	3.1250	+25 %
`AplicarAumentoGeneral(1.0)`	2.0	5.000	+100 %
`AplicarAumentoGeneral(10)`	11.0	27.500	+1000 % (no +10 %)

```
PROCEDIMIENTO AplicarAumentoGeneral(porcentaje)
 SI (porcentaje no es finito) ENTONCES ESCRIBIR "..." ; RETORNAR 0
 SI (porcentaje < 0) ENTONCES ESCRIBIR "..." ; RETORNAR 0
 SI (porcentaje == 0.0) ENTONCES ESCRIBIR "..." ; RETORNAR 0
 factor = 1 + porcentaje
 actual = inicio ; modificados = 0
 CICLO (actual != null)
 med = actual.dato
```

```
med.setPrecio(med.precio * factor)
modificados++
actual = actual.siguiente
RETORNAR modificados
```

Complejidad. Tiempo O(n); memoria adicional O(1).

Casos cubiertos. +10 % sobre 5 medicamentos · negativo · cero · lista vacía · un solo nodo · NaN · infinito.

Resumen de complejidades

Método	Tiempo	Memoria adicional
a) MedicamentosAgotados	O(n)	O(k) (arreglo resultado)
b) PromedioPrecioPorCategoria	O(n)	O(1)
c) ActualizarStock	O(n)	O(1)
d) ListarPreciosInversos	O(n)	O(1) + arreglo de salida O(n)
e) AplicarAumentoGeneral	O(n)	O(1)

6. Pruebas y resultados

`Main.java` es un *test harness* manual: por cada caso imprime el estado ANTES, la LLAMADA, el resultado ESPERADO, el estado DESPUÉS y el RESULTADO obtenido. Cada bloque construye sus propios datos en un ámbito independiente, de modo que ninguna operación depende del estado que deje otra. No se usa un framework (JUnit) ni aserciones automáticas: la verificación consiste en comparar visualmente *Esperado* con *Resultado*.

Datos de prueba base

Código	Nombre	Categoría	Stock	Precio	¿Agotado?
M01	Paracetamol	ANALGÉSICO	15	2.50	No
M02	Amoxicilina	ANTIBIÓTICO	3	12.00	Sí
M03	Ibuprofeno	ANTIINFLAMATORIO	2	4.50	Sí
M04	Vitamina C	VITAMINA	20	5.00	No
M05	Azitromicina	ANTIBIÓTICO	5	15.00	Sí

Cobertura por operación (27 casos)

Operación	Positivos	Negativos	Borde	Total
a) MedicamentosAgotados	1	2	1	4
b) PromedioPrecioPorCategoria	2	3	—	5
c) ActualizarStock	2	4	1	7
d) ListarPreciosInversos	1	1	2	4
e) AplicarAumentoGeneral	1	5	1	7
Total	**7**	**15**	**5**	**27**

Alcance de las pruebas

La batería cubre flujos positivos, negativos y borde de las cinco operaciones, incluyendo lista vacía (en las cinco) y elemento no encontrado (código inexistente en c, categoría sin medicamentos en b). Las validaciones de `Medicamento` (stock negativo, precio negativo, campos `null`) existen en el constructor y los *setters*, pero **no tienen casos propios** en `Main`: las pruebas se centran en el comportamiento de las operaciones de la lista.

Requisito de la guía oficial	Cómo se cubre
Proyecto Java funcional, orientado a objetos, con la estructura asignada	Lista simplemente enlazada en `FarmaVida/src` (cinco clases).
Casos de prueba positivos y negativos (listas vacías, no encontrado)	27 casos: 7 positivos, 15 negativos y 5 borde; ver tabla anterior y salida de ejecución.
Datos de prueba sugeridos (M01–M05)	Son exactamente los datos base de la sección 6.
README con problemática, estructura, casos ejecutados y declaración de IA	`README.md` del repositorio.
Declaración explícita de IA, indicando en qué partes	Sección 7 de este informe y sección homónima del README.

Verificación. Para este informe se compiló el código del repositorio con `javac -encoding UTF-8` y se ejecutó con OpenJDK 21: los 27 casos terminaron sin excepciones y en todos el *Resultado* coincide con lo *Esperado*. A continuación se reproduce la salida real.

Salida de la ejecución de `Main`

```
=== a) MedicamentosAgotados() ===

--- Caso positivo: algunos medicamentos agotados ---
ANTES : M01 → M02 → M03 → M04 → M05
Llamada: MedicamentosAgotados()
Esperado: [Amoxicilina, Ibuprofeno, Azitromicina]
DESPUÉS: M01 → M02 → M03 → M04 → M05 (sin cambios)
Resultado: [Amoxicilina, Ibuprofeno, Azitromicina]

--- Caso negativo: lista vacía ---
ANTES : (lista vacía)
Llamada: MedicamentosAgotados()
Esperado: []
DESPUÉS: (lista vacía, sin cambios)
Resultado: []

--- Caso negativo: ningún medicamento con stock <= 5 ---
ANTES : X01(stock=10) → X02(stock=20) → X03(stock=15)
Llamada: MedicamentosAgotados()
Esperado: []
DESPUÉS: X01(stock=10) → X02(stock=20) → X03(stock=15) (sin cambios)
Resultado: []

--- Caso borde: todos los medicamentos agotados ---
ANTES : Y01(stock=3) → Y02(stock=5) → Y03(stock=1)
Llamada: MedicamentosAgotados()
Esperado: [MedD, MedE, MedF]
DESPUÉS: Y01(stock=3) → Y02(stock=5) → Y03(stock=1) (sin cambios)
Resultado: [MedD, MedE, MedF]
```

```
=== b) PromedioPrecioPorCategoria() ===

--- Caso positivo: categoría con 2 medicamentos ---
ANTES : M01 → M02 → M03 → M04 → M05
Llamada: PromedioPrecioPorCategoria("ANTIBIÓTICO")
Esperado: 13.50
DESPUÉS: M01 → M02 → M03 → M04 → M05 (sin cambios)
Resultado: 13.50
```

```
--- Caso positivo: categoría con 1 medicamento ---
ANTES : M01 → M02 → M03 → M04 → M05
Llamada: PromedioPrecioPorCategoria("ANALGÉSICO")
Esperado: 2.50
DESPUÉS: M01 → M02 → M03 → M04 → M05 (sin cambios)
Resultado: 2.50
```

```
--- Caso negativo: categoría sin medicamentos ---
ANTES : M01 → M02 → M03 → M04 → M05
Llamada: PromedioPrecioPorCategoria("CATEGORIA_INEXISTENTE")
Esperado: 0.0
DESPUÉS: M01 → M02 → M03 → M04 → M05 (sin cambios)
Resultado: 0.0
```

```
--- Caso negativo: lista vacía ---
ANTES : (lista vacía)
Llamada: PromedioPrecioPorCategoria("ANTIBIÓTICO")
Esperado: 0.0
DESPUÉS: (lista vacía, sin cambios)
Resultado: 0.0
```

```
--- Caso negativo: categoría null ---
ANTES : M01 → M02 → M03 → M04 → M05
Llamada: PromedioPrecioPorCategoria(null)
Esperado: 0.0
DESPUÉS: M01 → M02 → M03 → M04 → M05 (sin cambios)
Resultado: 0.0
```

```
=== c) ActualizarStock() ===
```

```
--- Caso positivo: código existente (M02, 3 + 10) ---
ANTES : M02 (stock=3)
Llamada: ActualizarStock("M02", 10)
Esperado: true, stock después = 13
DESPUÉS: M02 (stock=13)
Resultado: true
```

```
--- Caso positivo: cantidad cero ---
ANTES : M03 (stock=2)
Llamada: ActualizarStock("M03", 0)
Esperado: true, stock después = 2
DESPUÉS: M03 (stock=2)
Resultado: true
```

```
--- Caso negativo: código inexistente ---
ANTES : M01 → M02 → M03 → M04 → M05
Llamada: ActualizarStock("M99", 10)
Esperado: false
Medicamento no encontrado
DESPUÉS: M01 → M02 → M03 → M04 → M05 (sin cambios)
Resultado: false
```

```
--- Caso negativo: cantidad negativa ---
ANTES : M02 (stock=13)
Llamada: ActualizarStock("M02", -5)
Esperado: false, stock sin cambios
La cantidad es negativa, se rechaza la solicitud
DESPUÉS: M02 (stock=13)
Resultado: false
```

```
--- Caso negativo: código null ---
ANTES : M01 → M02 → M03 → M04 → M05
Llamada: ActualizarStock(null, 10)
Esperado: false
El código es null, se rechaza la solicitud
DESPUÉS: M01 → M02 → M03 → M04 → M05 (sin cambios)
Resultado: false
```

```
--- Caso negativo: lista vacía ---
ANTES : (lista vacía)
Llamada: ActualizarStock("M01", 10)
Esperado: false
Medicamento no encontrado
DESPUÉS: (lista vacía, sin cambios)
Resultado: false
```

```
--- Caso borde: la suma excede el stock máximo ---
ANTES : MMAX (stock=2147483647)
Llamada: ActualizarStock("MMAX", 1)
Esperado: false, stock sin cambios
El stock excede el máximo permitido, se rechaza la solicitud
DESPUÉS: MMAX (stock=2147483647)
Resultado: false
```

```
=== d) ListarPreciosInversos() ===
```

```
--- Caso positivo: lista con 5 medicamentos ---
ANTES : M01(2.50) → M02(12.00) → M03(4.50) → M04(5.00) → M05(15.00)
Llamada: ListarPreciosInversos()
Esperado: [15.00, 5.00, 4.50, 12.00, 2.50]
DESPUÉS: M01(2.50) → M02(12.00) → M03(4.50) → M04(5.00) → M05(15.00) (restaurada)
Resultado: [15.00, 5.00, 4.50, 12.00, 2.50]
```

```
--- Caso negativo: lista vacía ---
ANTES : (lista vacía)
Llamada: ListarPreciosInversos()
Esperado: []
DESPUÉS: (lista vacía, sin cambios)
Resultado: []
```

```
--- Caso borde: lista con un solo nodo ---
ANTES : Z01(7.25)
Llamada: ListarPreciosInversos()
Esperado: [7.25]
DESPUÉS: Z01(7.25) (restaurada)
Resultado: [7.25]
```

```
--- Caso borde: el atributo cantidad no coincide con los nodos ---
ANTES : Q01(1.00) → Q02(2.00), cantidad=1
Llamada: ListarPreciosInversos()
Esperado: [2.00, 1.00], lista restaurada
DESPUÉS: lista restaurada = true
Resultado: [2.00, 1.00]
```

```
=== e) AplicarAumentoGeneral() ===
```

```
--- Caso positivo: aumento del 10% ---
ANTES : M01(2.50) → M02(12.00) → M03(4.50) → M04(5.00) → M05(15.00)
Llamada: AplicarAumentoGeneral(0.10)
Esperado: 5
DESPUÉS: M01(2.75) → M02(13.20) → M03(4.95) → M04(5.50) → M05(16.50)
Resultado: 5
```

```
--- Caso negativo: porcentaje negativo ---
ANTES : M01 → M02 → M03 → M04 → M05
Llamada: AplicarAumentoGeneral(-0.10)
Esperado: 0
El porcentaje no puede ser negativo
DESPUÉS: M01 → M02 → M03 → M04 → M05 (sin cambios)
Resultado: 0
```

```
--- Caso negativo: porcentaje cero ---
ANTES : M01 → M02 → M03 → M04 → M05
Llamada: AplicarAumentoGeneral(0.0)
Esperado: 0
No hubo cambios: el porcentaje es 0
DESPUÉS: M01 → M02 → M03 → M04 → M05 (sin cambios)
Resultado: 0
```

```
--- Caso negativo: lista vacía ---
ANTES : (lista vacía)
Llamada: AplicarAumentoGeneral(0.10)
Esperado: 0
DESPUÉS: (lista vacía, sin cambios)
Resultado: 0
```

```
--- Caso borde: lista con un solo nodo ---
ANTES : Z01(7.25)
Llamada: AplicarAumentoGeneral(0.10)
Esperado: 1
DESPUÉS: Z01(7.975)
Resultado: 1
```

```
--- Caso negativo: porcentaje NaN ---
Llamada: AplicarAumentoGeneral(Double.NaN)
Esperado: 0
El porcentaje debe ser un número finito
DESPUÉS: precios sin cambios
Resultado: 0

--- Caso negativo: porcentaje infinito ---
Llamada: AplicarAumentoGeneral(Double.POSITIVE_INFINITY)
Esperado: 0
El porcentaje debe ser un número finito
DESPUÉS: precios sin cambios
Resultado: 0
```

7. Declaración de uso de Inteligencia Artificial

En cumplimiento de las normas éticas y las indicaciones del curso, el equipo declara de forma transparente el uso de asistentes de IA como herramientas de apoyo académico. Las herramientas empleadas fueron **Claude (Anthropic), DeepSeek y Gemini**.

Dónde se usó la IA

Área	Uso de la IA	Control del equipo
Casos borde	Apoyo para identificar situaciones complejas previas a la implementación: desbordamiento de enteros en el stock, valores `NaN` e infinitos en `double`, división por cero.	Cada integrante decidió qué casos tratar y cómo.
Pseudocódigo	Revisión socrática: discusión de la lógica que cada integrante había diseñado antes a mano.	El diseño y el trazado de punteros son del equipo.
Documentación técnica	Apoyo en la estructura y la corrección de estilo de los documentos de lógica y de este informe.	El equipo revisó el contenido contra el código.
`Main.java` (batería de pruebas)	Se utilizó IA como apoyo en la elaboración de la batería de pruebas manual.	El equipo ejecutó las pruebas y comprobó que el resultado coincide con el esperado de cada método.
Guías de `BigDecimal` y `enum`	Se utilizó IA como apoyo en la elaboración de las dos guías de referencia de la carpeta `Documentación/`.	El equipo las usó como material de consulta durante el desarrollo.
Refactorización del código	No se declara uso de IA en la refactorización.	—
README e informe (PDF) actualizados, octubre de 2026	Redactados con **Claude (Anthropic)** a partir del código del repositorio, del informe anterior y de la ejecución real de las pruebas; el PDF se generó mediante un script con ese contenido. Incluye la documentación de las decisiones de diseño de la sección 4.	El equipo revisó y aprobó el resultado.

Qué no hizo la IA

- No se utilizó para generar código a ciegas ni para sustituir el aprendizaje y el razonamiento del equipo.
- Las cinco operaciones de `ListaSimple` se diseñaron manualmente antes de codificarlas.

- Ningún resultado de la IA se aceptó sin verificarlo: se comprobó contra la compilación y la ejecución del código.

Alcance y medición

El uso de la IA fue de apoyo y **no se mide de forma cuantitativa** (no se registraron porcentajes ni número de consultas). La tabla anterior describe en qué partes del proyecto intervino. El equipo es responsable del contenido final.

8. Reflexión sobre el proceso de trabajo

La metodología de *pensar y trazar a mano antes de programar* resultó muy efectiva. Dibujar el movimiento de `actual`, `anterior` y `siguiente` permitió detectar a tiempo errores que son difíciles de rastrear en ejecución, y razonar sobre la inmutabilidad de `BigDecimal` (las operaciones devuelven un objeto nuevo: hay que asignar el resultado) evitó cálculos que parecían correctos pero no modificaban nada.

Aprendizajes concretos:

- Un retorno ambiguo (`false`, `0`) obliga a informar la causa; por eso hay mensajes en `ActualizarStock` y `AplicarAumentoGeneral`.
- El tamaño de un arreglo resultado debe calcularse con el recorrido real, no con un atributo que puede desalinearse (`cantidad`).
- Comparar tipos distintos (enum contra String) compila sin error pero falla en silencio.
- Probar casos negativos y borde descubre más defectos que repetir casos felices.

La separación clara de responsabilidades entre los cinco integrantes, con pruebas independientes por operación, permitió trabajar en paralelo sin que el estado de una prueba afectara a otra. El uso de IA como interlocutor para cuestionar el diseño funcionó mejor que como generador de soluciones, y mantuvo el razonamiento en manos del equipo.

9. Conclusiones y trabajo futuro

El proyecto afianzó el dominio de las estructuras enlazadas en Java. Se obtuvo un sistema robusto y eficiente, protegido frente a errores habituales: desbordamiento de enteros, divisiones por cero, tipos incompatibles, valores no finitos y entradas `null`. Las cinco operaciones están implementadas, documentadas y verificadas con 27 casos que se ejecutan sin fallos.

Observaciones conocidas

- **Escala de los resultados:** `AplicarAumentoGeneral` no redondea (p. ej. 2.750), mientras que `PromedioPrecioPorCategoria` usa 2 decimales y devuelve `0.0` sin coincidencias.
- **Atributo** `cantidad`**:** la lista no tiene un método de inserción que lo actualice; se asigna con `setCantidad`. Ninguna operación depende de él.
- **Pruebas manuales:** la verificación es visual, sin aserciones automáticas.
- **Convención del porcentaje:** `AplicarAumentoGeneral(0.10)` aplica +10 %; `AplicarAumentoGeneral(10)` aplicaría +1000 % (sección 4).

- **Categoría exacta:** `PromedioPrecioPorCategoria` espera el nombre del enum en mayúsculas y con tilde (sección 4).
- **Excepciones de** `Medicamento` **:** están implementadas pero no tienen casos propios en la batería.

Anexo. Estructura del repositorio y ejecución

```
Parcial-1/
├── README.md
├── .gitignore / .gitattributes
├── Documentación/
│ ├── Lógica_ActualizarStock.md
│ ├── Lógica_AplicarAumentoGeneral.md
│ ├── Lógica_ListarPreciosInversos.md
│ ├── Lógica_MedicamentosAgotados.md
│ ├── Lógica_PromedioPrecioPorCategoria.md
│ ├── Guia_BigDecimal_Java.md
│ ├── Guia_enum_Java.md
│ └── Informe_FarmaVida.pdf
└── FarmaVida/
 ├── .vscode/settings.json
 └── src/ Categoria.java · Medicamento.java · Nodo.java · ListaSimple.java · Main.java
```

Compilar y ejecutar (JDK 8 o superior)

```
git clone https://github.com/RelaxUniver/Parcial-1.git
cd Parcial-1/FarmaVida
javac -encoding UTF-8 -d bin src/*.java
java -cp bin Main
```

Si las tildes o las flechas aparecen como «?», configure la consola en UTF-8 (en Windows: `chcp 65001`).